

1.Задание

Построить ER-диаграмму для концептуального этапа моделирования базы данных для выбранной предметной области – благотворительный фонд (БФ) помощи животным «Хвостик».

2. Ход работы

1.1 Обзор предметной области.

Предметная область – благотворительный фонд “Хвостик”. Фонд предназначен для помощи животным, посредством организации денежных сборов, помощи приютам, проведения мероприятия, привлекающих благотворителей, а также оказание различных ветеринарных услуг по льготной стоимости гражданам РФ.

1.2 Сущности и связи

В ходе работы были выделены следующие сущности и их атрибуты:

1.2.1 Сборы - основная сущность нашей предметной области.

Имеет следующие атрибуты:

А) Id сбора - уникальный код сбора.

Б) Сумма – целевая сумма сбора.

В) Цель – цель сбора.

1.2.2 Благотворители фонда - основная сущность нашей предметной области.

Имеет следующие атрибуты:

А) ФИО

Б) Паспортные данные

В) Дата рождения

Г) Пол

1.2.3 Мероприятия - вспомогательная сущность для благотворителя.

Имеет следующие атрибуты:

А) Id мероприятия - уникальный код мероприятия, на котором был привлечен благотворитель.

Б) Дата события.

В) Название мероприятия.

Присутствует связь между благотворителем и сбором – делает пожертвование.

1.3 Ограничения, накладываемые на область

- сумма сбора должна составлять не менее 10.000.
- дата событие должна быть не раньше 01.06.2015.
- Благотворителями являются люди, имеющие паспорт РФ.
- сумма донатов может превышать сумму сбора.

1.4 Диаграмма

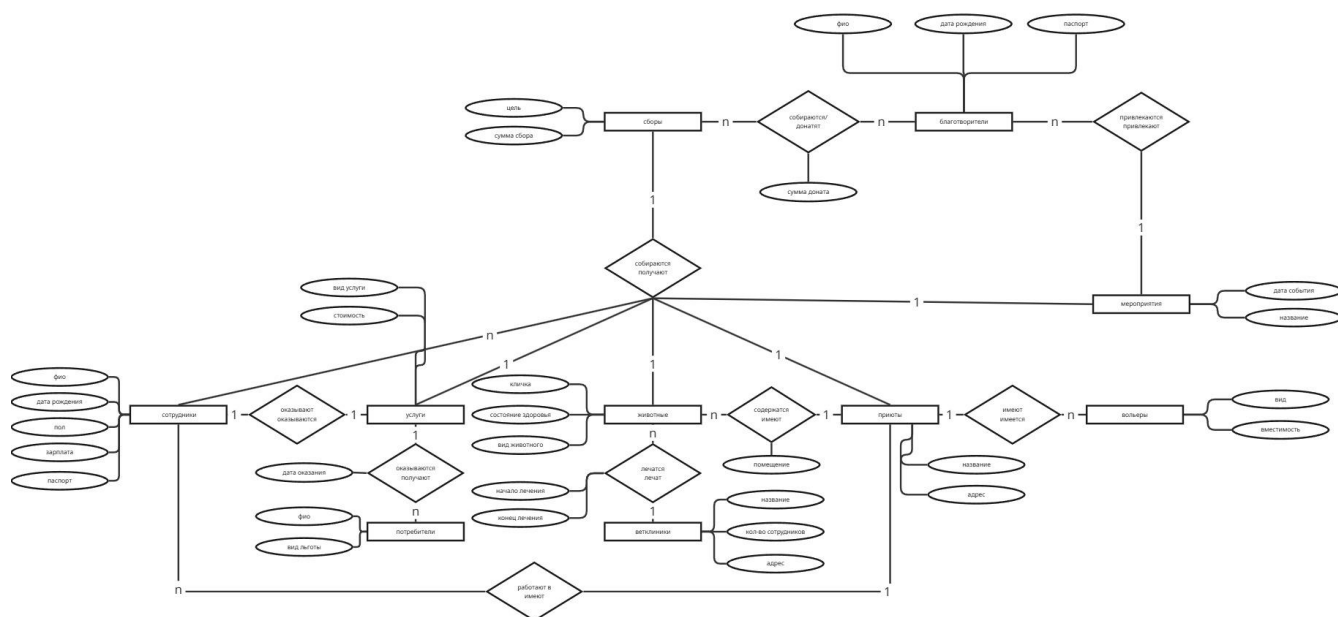


Рис. 1: ER-диаграмма базы данных БФ «Хвостик».

2. Логическая модель

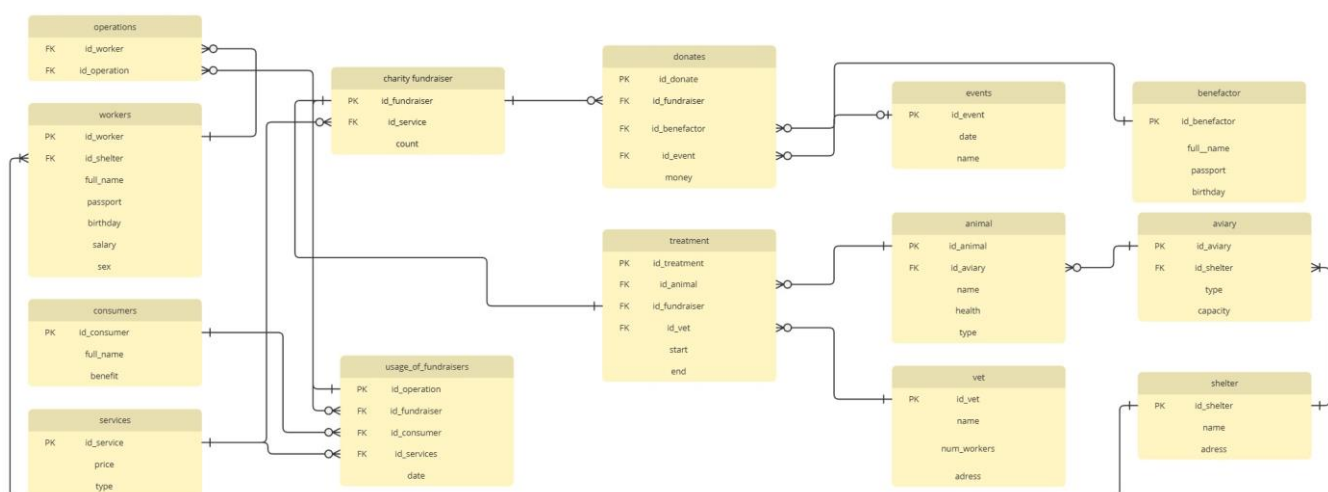


Рис. 2: Диаграмма логического этапа проектирования в нотации «crow's foot»

3. Физическая модель

В качестве СУБД для реализации разработанной базы данных была выбрана PostgreSQL.

В связи с проведенным анализом предметной области была проработана следующая физическая схема БД.

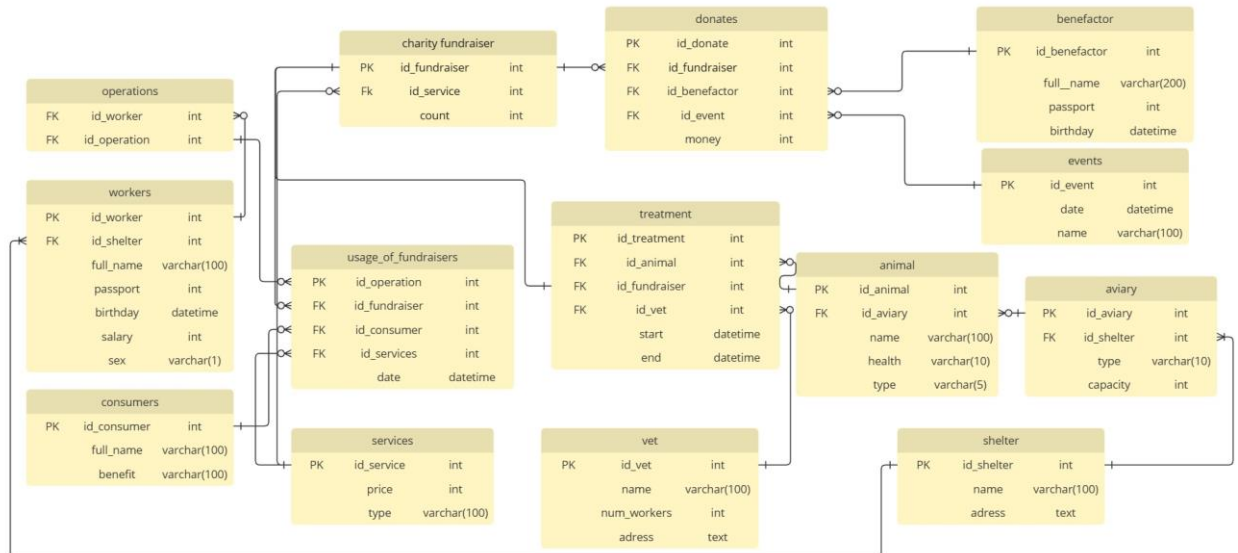


Рис. 3: Физическая модель БД БФ «Хвостик».

4. Создание таблиц

Ниже приведены запросы на языке SQL (диалект PostgreSQL) для создания таблицы, описанных выше.

- Код для создания таблицы “shelter”

```
CREATE TABLE shelter (  
    id_shelter SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    adress TEXT NOT NULL  
);
```

Рис. 4: Код для создания таблицы “shelter”

- Код для создания таблицы “aviary”

```
CREATE TABLE aviary (  
    id_aviary SERIAL PRIMARY KEY,  
    id_shelter INT NOT NULL REFERENCES shelter(id_shelter) ON DELETE CASCADE,  
    type VARCHAR(10) NOT NULL,  
    capacity INT NOT NULL  
);
```

Рис. 5: Код для создания таблицы “aviary”

- Код для создания таблицы “animal”

```
CREATE TABLE animal (  
    id_animal SERIAL PRIMARY KEY,  
    id_aviary INT NOT NULL REFERENCES aviary(id_aviary) ON DELETE CASCADE,  
    name VARCHAR(100),  
    health VARCHAR(10)  
);
```

Рис. 6: Код для создания таблицы “animal”

- Код для создания таблицы “vet”

```
CREATE TABLE vet (  
    id_vet SERIAL PRIMARY KEY,  
    name VARCHAR(100) NOT NULL,  
    num_workers INT NOT NULL,  
    adress TEXT NOT NULL  
);
```

Рис. 7: Код для создания таблицы “vet”

- Код для создания таблицы “workers”

```
CREATE TABLE workers (  
    id_worker SERIAL PRIMARY KEY,  
    id_shelter INT NOT NULL REFERENCES shelter(id_shelter) ON DELETE CASCADE,  
    full_name VARCHAR(100) NOT NULL,  
    passport INT UNIQUE NOT NULL,  
    birthday DATE NOT NULL,  
    salary INT NOT NULL,  
    sex VARCHAR(1)  
);
```

Рис. 8: Код для создания таблицы “workers”

- Код для создания таблицы “operations”

```
CREATE TABLE operations (  
    id_operation SERIAL PRIMARY KEY,  
    id_worker INT NOT NULL REFERENCES workers(id_worker) ON DELETE CASCADE  
);
```

Рис. 9: Код для создания таблицы “operations”

- Код для создания таблицы “consumers”

```
CREATE TABLE consumers (
    id_consumer SERIAL PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    benefit VARCHAR(100) NOT NULL
);
```

Рис. 10: Код для создания таблицы “consumers”

- Код для создания таблицы “services”

```
CREATE TABLE services (
    id_service SERIAL PRIMARY KEY,
    price INT NOT NULL,
    type VARCHAR(100) NOT NULL
);
```

Рис. 11: Код для создания таблицы “services”

- Код для создания таблицы “charity_fundraiser”

```
CREATE TABLE charity_fundraiser (
    id_fundraiser SERIAL PRIMARY KEY,
    id_service INT NOT NULL REFERENCES services(id_service) ON DELETE CASCADE,
    count INT NOT NULL
);
```

Рис. 12: Код для создания таблицы “charity_fundraiser”

- Код для создания таблицы “usage_of_fundraisers”

```
CREATE TABLE usage_of_fundraisers (
    id_operation INT NOT NULL REFERENCES operations(id_operation) ON DELETE CASCADE,
    id_fundraiser INT NOT NULL REFERENCES charity_fundraiser(id_fundraiser) ON DELETE CASCADE,
    id_consumer INT NOT NULL REFERENCES consumers(id_consumer) ON DELETE CASCADE,
    id_services INT NOT NULL REFERENCES services(id_service) ON DELETE CASCADE,
    date TIMESTAMP ,
    PRIMARY KEY (id_operation, id_fundraiser)
);
```

Рис. 13: Код для создания таблицы “usage_of_fundraisers”

- Код для создания таблицы “events”

```
CREATE TABLE events (
    id_event SERIAL PRIMARY KEY,
    date TIMESTAMP NOT NULL,
    name VARCHAR(100)
);
```

Рис. 14: Код для создания таблицы “events”

- Код для создания таблицы “benefactor”

```
CREATE TABLE benefactor (  
    id_benefactor SERIAL PRIMARY KEY,  
    full_name VARCHAR(200) NOT NULL,  
    passport INT UNIQUE NOT NULL,  
    birthday DATE NOT NULL,  
    sex VARCHAR(1)  
);
```

Рис. 15: Код для создания таблицы “benefactor”

- Код для создания таблицы “donates”

```
CREATE TABLE donates (  
    id_donate SERIAL PRIMARY KEY,  
    id_fundraiser INT NOT NULL REFERENCES charity_fundraiser(id_fundraiser) ON DELETE CASCADE,  
    id_benefactor INT NOT NULL REFERENCES benefactor(id_benefactor) ON DELETE CASCADE,  
    id_event INT NOT NULL REFERENCES events(id_event) ON DELETE CASCADE,  
    money INT NOT NULL  
);
```

Рис. 16: Код для создания таблицы “donates”

- Код для создания таблицы “treatment”

```
CREATE TABLE treatment (  
    id_treatment SERIAL PRIMARY KEY,  
    id_animal INT NOT NULL REFERENCES animal(id_animal) ON DELETE CASCADE,  
    id_fundraiser INT NOT NULL REFERENCES charity_fundraiser(id_fundraiser) ON DELETE CASCADE,  
    id_vet INT NOT NULL REFERENCES vet(id_vet) ON DELETE CASCADE,  
    start TIMESTAMPTZ NOT NULL,  
    "end" TIMESTAMPTZ  
);
```

Рис. 17: Код для создания таблицы “treatment”

5. Заполнение таблиц

Далее представлены результаты работы программы на примере таблиц, соответствующих функциям, приведенным выше.

- Таблица treatment

```
treatment_rows = []
for _ in range(10000):
    start_date = fake.date_between(start_date="-10y", end_date="today")

    if random.random() < 0.05:
        end_date = None
    else:
        end_date = start_date + timedelta(days=random.randint(1, 30))

    treatment_rows.append((
        random.choice(animals_ids),
        random.choice(fundraiser_ids),
        random.choice(vet_ids),
        start_date,
        end_date
    ))
insert_many(
    "INSERT INTO treatment(id_animal, id_fundraiser, id_vet, start, \"end\") VALUES (%s, %s, %s, %s, %s)",
    treatment_rows
)
```

Рис. 18: Код для заполнения таблицы “ treatment”

	id_treatment [PK] integer	id_animal integer	id_fundraiser integer	id_vet integer	start timestamp without time zone	end timestamp without time zone
1	1	2002	730	25	2016-10-23 00:00:00	2016-11-12 00:00:00
2	2	433	3436	19	2018-10-01 00:00:00	2018-10-11 00:00:00
3	3	3337	1652	15	2018-04-14 00:00:00	2018-04-16 00:00:00
4	4	1731	3541	14	2019-10-07 00:00:00	2019-11-05 00:00:00
5	5	963	3045	20	2018-01-30 00:00:00	2018-02-23 00:00:00
6	6	1331	1484	9	2019-01-15 00:00:00	2019-02-02 00:00:00
7	7	4276	1327	12	2016-12-11 00:00:00	2016-12-19 00:00:00
8	8	3614	780	14	2024-10-12 00:00:00	2024-11-01 00:00:00
9	9	3715	2606	12	2022-04-14 00:00:00	2022-04-17 00:00:00
10	10	735	1986	7	2024-05-14 00:00:00	2024-06-10 00:00:00
11	11	887	3024	22	2021-01-19 00:00:00	[null]
12	12	1867	253	2	2021-05-08 00:00:00	2021-05-29 00:00:00
13	13	2727	4521	21	2018-11-12 00:00:00	2018-12-12 00:00:00
14	14	921	883	26	2021-12-10 00:00:00	2021-12-30 00:00:00
15	15	1805	2452	25	2021-05-02 00:00:00	2021-05-11 00:00:00
16	16	1125	2057	19	2018-10-24 00:00:00	2018-11-17 00:00:00
17	17	2764	3172	4	2020-03-30 00:00:00	2020-04-02 00:00:00

Рис. 19: Результат для заполнения таблицы “ treatment”

- Таблица donates

```
donates = []
for _ in range(10000):
    event_id = random.choice(event_ids)
    donates.append((
        random.choice(fundraiser_ids),
        random.choice(benefactor_ids),
        event_id,
        random.randint(100, 20000)
    ))
insert_many(
    "INSERT INTO donates(id_fundraiser, id_benefactor, id_event, money) VALUES (%s, %s, %s, %s)",
    donates
)
```

Рис. 20: Код для заполнения таблицы “ donates”

	id_donate [PK] integer	id_fundraiser integer	id_benefactor integer	id_event integer	money integer
1	1	153	2117	7	16063
2	2	574	4680	91	9648
3	3	3558	2692	6	14298
4	4	2779	4848	67	1359
5	5	1587	3639	8	235
6	6	3416	519	70	10015
7	7	1682	4826	55	2796
8	8	1162	3768	7	9714
9	9	1780	3858	23	16690
10	10	3655	3470	25	4617
11	11	907	2254	31	15556
12	12	761	1193	1	13003
13	13	148	3033	88	13433
14	14	1286	1056	91	13303
15	15	4817	1331	50	7761
16	16	1098	428	50	3177
17	17	2369	1159	30	3433
18	18	4174	4091	33	14424
19	19	256	1207	30	19503
20	20	3228	1074	65	16558

Рис. 21: Результат для заполнения таблицы “donates”

- Таблица benefactor

```
benefactor_rows = []
for _ in range(10000):
    sex = None if random.random() < 0.05 else random.choice(["M", "F"])
    if sex == "M":
        full_name = fake.first_name_male() + " " + fake.last_name_male()
    elif sex == "F":
        full_name = fake.first_name_female() + " " + fake.last_name_female()
    else:
        full_name = fake.first_name() + " " + fake.last_name()

    passport = fake.random_int(100000, 999999)
    while passport in global_passports:
        passport = fake.random_int(100000, 999999)
    global_passports.add(passport)

    benefactor_rows.append(
        (full_name, passport, fake.date_of_birth(minimum_age=18, maximum_age=95), sex)
    )

insert_many(
    "INSERT INTO benefactor(full_name, passport, birthday, sex) VALUES (%s, %s, %s, %s)",
    benefactor_rows
)

cur.execute("SELECT id_benefactor FROM benefactor")
benefactor_ids = [r[0] for r in cur.fetchall()]
```

Рис. 22: Код для заполнения таблицы “benefactor”

	id_benefactor [PK] integer	full_name character varying (200)	passport integer	birthday date	sex character varying (1)
1	1	Jack Douglas	334217	1970-06-27	M
2	2	Robert McCoy	347292	1977-01-11	M
3	3	Ariel Rich	658958	1955-03-20	F
4	4	Natalie Martinez	932500	1975-03-09	[null]
5	5	Joshua Gomez	728164	1985-11-07	[null]
6	6	Sherri Potter	661559	1982-11-07	F
7	7	Christopher Reeves	685187	1988-09-01	M
8	8	Lisa Moore	931237	1995-05-26	F
9	9	Heather Sims	323823	1995-12-10	F
10	10	Lori James	864507	1985-09-04	F
11	11	Paula Nelson	927303	1980-04-28	F
12	12	Stephanie McIntyre	266760	1999-03-20	F
13	13	Jacqueline Lopez	211101	1992-12-01	F
14	14	Daniel Harris	157898	1976-11-13	M
15	15	James Chambers	523329	2007-04-01	M
16	16	Lisa Anderson	658169	1999-03-13	F
17	17	Nathaniel Villarreal	746346	1995-06-11	M
18	18	Hannah French	881792	1951-04-14	F
19	19	Austin Johnson	628073	1954-09-02	M
20	20	Kimberly Johnson	820608	2001-08-25	F

Рис. 23: Результат для заполнения таблицы “benefactor”

- Таблица events

```

event_ids = []
years = list(range(2015, 2025))
total_events = 100

for _ in range(total_events):
    name = random.choice(event_names)
    year = random.choice(years)
    month = random.randint(1, 12)
    day = random.randint(1, 28)

    date = f"{year}-{month:02d}-{day:02d}"

    cur.execute(
        "INSERT INTO events(date, name) VALUES (%s, %s) RETURNING id_event",
        (date, name)
    )
    event_ids.append(cur.fetchone()[0])
conn.commit()

```

Рис. 24: Код для заполнения таблицы “events”

	id_event [PK] integer	date timestamp without time zone	name character varying (100)
1	1	2019-04-12 00:00:00	Health Day
2	2	2015-10-16 00:00:00	Volunteer Day
3	3	2018-04-23 00:00:00	Charity Day
4	4	2018-02-09 00:00:00	Health Day
5	5	2023-11-19 00:00:00	Animal Protection Day
6	6	2016-09-22 00:00:00	Health Day
7	7	2016-11-16 00:00:00	Health Day
8	8	2021-09-14 00:00:00	Charity Day
9	9	2019-06-22 00:00:00	Volunteer Day
10	10	2021-05-22 00:00:00	Shelter Day
11	11	2017-01-14 00:00:00	Animal Protection Day
12	12	2020-06-17 00:00:00	Animal Protection Day
13	13	2015-01-07 00:00:00	Volunteer Day
14	14	2023-01-22 00:00:00	Charity Day
15	15	2020-07-21 00:00:00	another
16	16	2019-12-10 00:00:00	Volunteer Day
17	17	2015-09-17 00:00:00	Christmas

Рис. 25: Результат для заполнения таблицы “events”

- Таблица usage_of_fundraisers

```
usage_set = set()
usage_rows = []
while len(usage_rows) < 10000:
    op = random.choice(operation_ids)
    fr = random.choice(fundraiser_ids)

    if (op, fr) in usage_set:
        continue

    usage_set.add((op, fr))
    date = None if random.random() < 0.05 else fake.date_time_between(
        start_date="-10y",
        end_date="now"
    )
    usage_rows.append((
        op,
        fr,
        random.choice(consumer_ids),
        random.choice(service_ids),
        date
    ))
insert_many("""
INSERT INTO usage_of_fundraisers
(id_operation, id_fundraiser, id_consumer, id_services, date)
VALUES (%s, %s, %s, %s, %s)

```

Рис. 26: Код для заполнения таблицы “usage_of_fundraisers”

	id_operation [PK] integer	id_fundraiser [PK] integer	id_consumer integer	id_services integer	date timestamp without time zone
1	2965	3637	1286	7	2016-06-23 18:04:59
2	4006	3825	948	6	[null]
3	2750	2526	786	7	2025-10-07 07:13:01
4	3822	4089	1499	6	2023-09-01 07:37:40
5	3390	38	4803	4	2025-11-04 05:30:18
6	4162	2788	4521	6	2024-05-08 02:01:51
7	4392	4509	515	1	2018-02-08 08:49:17
8	4291	2811	3886	2	2022-11-01 02:26:05
9	2841	4104	3923	7	2018-03-04 20:21:32
10	4843	2404	1594	7	2023-05-17 14:32:52
11	3365	2244	4065	1	2025-10-17 02:31:03
12	1379	3135	4015	4	2019-07-29 01:20:32
13	4037	3476	2873	5	2021-12-19 01:16:11
14	3226	2162	4246	4	2022-07-07 20:00:15
15	3173	3155	4779	6	2019-02-03 03:57:49
16	2172	3626	2858	3	2020-09-22 23:49:43
17	1955	2254	2071	7	2022-05-20 06:38:41
18	2620	296	1063	3	2020-01-18 19:12:47
19	193	2987	4057	2	2023-02-11 00:28:04
20	4920	46	2236	6	2023-07-17 17:48:09

Рис. 27: Результат для заполнения таблицы “usage_of_fundraisers”

- Таблица charity_fundraiser

```

fundraisers = [
    (random.choice(service_ids), random.randint(1, 50))
    for _ in range(5000)
]

insert_many(
    "INSERT INTO charity_fundraiser(id_service, count) VALUES (%s, %s)",
    fundraisers
)

cur.execute("SELECT id_fundraiser FROM charity_fundraiser")
fundraiser_ids = [r[0] for r in cur.fetchall()]

```

Рис. 28: Код для заполнения таблицы “charity_fundraiser”

	id_fundraiser [PK] integer	id_service integer	count integer
1	1	5	47
2	2	7	47
3	3	5	42
4	4	3	37
5	5	4	19
6	6	5	40
7	7	4	40
8	8	5	4
9	9	3	11
10	10	7	9
11	11	1	32
12	12	1	8
13	13	3	35
14	14	7	27
15	15	7	22
16	16	4	23
17	17	5	48

Рис. 29: Результат для заполнения таблицы “charity_fundraiser”

- Таблица services

```
service_ids = []
for stype in service_types:
    price = random.randint(100, 2000)
    cur.execute(
        "INSERT INTO services(price, type) VALUES (%s, %s) RETURNING id_service",
        (price, stype)
    )
    service_ids.append(cur.fetchone()[0])
conn.commit()
```

Рис. 30: Код для заполнения таблицы “services”

	id_service [PK] integer	price integer	type character varying (100)
1	1	1836	treatment
2	2	1794	shelter_support
3	3	1083	food_payment
4	4	785	medicines
5	5	1486	warehouse_payment
6	6	1017	vet_clinic_payment
7	7	1176	another

Рис. 31: Результат для заполнения таблицы “services”

- Таблица consumers

```
consumers = [(fake.name(), random.choice(consumer_benefits)) for _ in range(10000)]
insert_many("INSERT INTO consumers(full_name, benefit) VALUES (%s, %s)", consumers)

cur.execute("SELECT id_consumer FROM consumers")
consumer_ids = [r[0] for r in cur.fetchall()]
```

Рис. 32: Код для заполнения таблицы “consumers”

	id_consumer [PK] integer	full_name character varying (100)	benefit character varying (100)
1	1	Jamie Hays	low_income
2	2	Christopher Walters	low_income
3	3	James Bailey	another
4	4	Ashley Mills	volunteer
5	5	Robert Allen	low_income
6	6	Catherine Reyes	shelter
7	7	Shane Ford	shelter
8	8	Bruce Chan	another
9	9	Leslie Hawkins	another
10	10	Samantha Olson	shelter
11	11	Matthew Hanson DDS	low_income
12	12	Misty Richard	another
13	13	Aaron Walker	volunteer
14	14	Mr. Eric Simmons	volunteer
15	15	Douglas Davis	volunteer
16	16	Jeremy Kane	low_income
17	17	Peter Johns	shelter

Рис. 33: Результат для заполнения таблицы “consumers”

- Таблица operations

```
operations = [(random.choice(worker_ids),) for _ in range(10000)]
insert_many("INSERT INTO operations(id_worker) VALUES (%s)", operations)

cur.execute("SELECT id_operation FROM operations")
operation_ids = [r[0] for r in cur.fetchall()]
```

Рис. 34: Код для заполнения таблицы “operations”

	id_operation [PK] integer	id_worker integer
1	1	58
2	2	22
3	3	4
4	4	13
5	5	3
6	6	36
7	7	25
8	8	23
9	9	21
10	10	30
11	11	6
12	12	23
13	13	26
14	14	27
15	15	26
16	16	17
17	17	38
18	18	59

Рис. 35: Результат для заполнения таблицы “operations”

- Таблица workers

```

worker_ids = []
for _ in range(10000):
    sex = random.choice(["M", "F"])
    full_name = fake.first_name_male() + " " + fake.last_name_male() if sex == "M" \
        else fake.first_name_female() + " " + fake.last_name_female()

    passport = fake.random_int(100000, 999999)
    while passport in global_passports:
        passport = fake.random_int(100000, 999999)
    global_passports.add(passport)

    cur.execute(
        """INSERT INTO workers(id_shelter, full_name, passport, birthday, salary, sex)
        VALUES (%s, %s, %s, %s, %s, %s) RETURNING id_worker"""
    )
    (
        random.choice(shelter_ids),
        full_name,
        passport,
        fake.date_of_birth(minimum_age=18, maximum_age=60),
        random.randint(30000, 90000),
        sex
    )
)
worker_ids.append(cur.fetchone()[0])
conn.commit()

```

Рис. 36: Код для заполнения таблицы “workers”

	id_worker [PK] integer	id_shelter integer	full_name character varying (100)	passport integer	birthday date	salary integer	sex character varying (1)
1	1	7	Kevin Moore	949283	1990-09-...	37009	M
2	2	4	Catherine Gardner	303365	1992-08-...	59299	F
3	3	3	Michael Cross	479847	1976-08-...	77468	M
4	4	6	John Williams	404508	1966-07-...	51306	M
5	5	7	Patricia Stone	132339	1995-04-...	73222	F
6	6	8	Jessica Schmidt	961007	1974-03-...	73458	F
7	7	3	James Gibson	439350	1995-12-...	43551	M
8	8	6	Karen Banks	854530	1965-06-...	42320	F
9	9	8	Kyle Torres	161967	1989-09-...	62041	M
10	10	7	Amy Pena	354083	2003-12-...	33206	F
11	11	1	Connie Mendoza	160647	1966-02-...	58396	F
12	12	3	Victoria Wood	800354	1977-09-...	34024	F
13	13	8	Ross Mathis	313812	1976-03-...	64359	M
14	14	4	Nancy Collins	855548	1975-05-...	57092	F
15	15	5	Brandon Miller	657591	1991-07-...	39220	M
16	16	5	Tanya Stout	318919	1974-03-...	41603	F
17	17	1	Lisa Hood	366341	1988-07-...	61013	F

Рис. 37: Результат для заполнения таблицы “workers”

- Таблица vet

```

vet_ids = []
for _ in range(100):
    cur.execute(
        "INSERT INTO vet(name, num_workers, adress) VALUES (%s, %s, %s) RETURNING id_vet",
        (random.choice(vet_names), random.randint(1, 30), fake.address())
    )
    vet_ids.append(cur.fetchone()[0])
conn.commit()

```

Рис. 38: Код для заполнения таблицы “vet”

	id_vet [PK] integer	name character varying (100)	num_workers integer	adress text
1	1	Home Vet	12	09663 Ethan Causeway Suite 970
2	2	Animal Health Center	13	94468 Lindsey Trail
3	3	Vet Doctor	14	9716 Tanya Parkway Apt. 820
4	4	Vet Doctor	10	11755 James Station
5	5	Home Vet	8	56903 Shields Greens Apt. 915
6	6	Vet Doctor	4	766 Krause Spurs
7	7	Animal Clinic	14	85828 Gregg Lodge
8	8	Home Vet	20	447 Day Parks
9	9	ZooMed Clinic	1	0507 Lauren Roads
10	10	Vet Clinic #1	13	Unit 1031 Box 1951
11	11	Vet Doctor	7	80035 Paul Motorway
12	12	Animal Clinic	9	98776 Glover Radial
13	13	Animal Clinic	5	0449 Zachary Throughway
14	14	Vet Doctor	6	27905 Anderson Meadow
15	15	Animal Clinic	19	7461 Hudson Dam

Рис. 39: Результат для заполнения таблицы “vet”

- Таблица animal

```

cur.execute("SELECT id_aviary, capacity FROM aviary")
aviary_capacity = {row[0]: row[1] for row in cur.fetchall()}

total_capacity = sum(aviary_capacity.values())
num_animals = total_capacity - 1500

animal_ids = []
animals_to_insert = []

for aviary_id, cap in aviary_capacity.items():
    count = random.randint(0, cap)

    for _ in range(count):
        if len(animals_to_insert) >= num_animals:
            break
        name = None if random.random() < 0.05 else fake.first_name()
        health = None if random.random() < 0.05 else random.choice(["healthy", "sick", "average"])
        animals_to_insert.append((aviary_id, name, health))

insert_many(
    "INSERT INTO animal(id_aviary, name, health) VALUES (%s, %s, %s)",
    animals_to_insert
)
cur.execute("SELECT id_animal FROM animal")
animal_ids = [r[0] for r in cur.fetchall()]

```

Рис. 40: Код для заполнения таблицы “animal”

	id_animal [PK] integer	id_aviary integer	name character varying (100)	health character varying (10)
1	1	85	William	healthy
2	2	99	Carlos	healthy
3	3	87	Cynthia	sick
4	4	15	Corey	healthy
5	5	34	Raven	average
6	6	94	[null]	sick
7	7	15	Steven	average
8	8	6	Joseph	sick
9	9	104	Cheryl	[null]
10	10	95	James	average
11	11	86	Kimberly	healthy
12	12	109	Michael	average
13	13	40	Veronica	average
14	14	38	Linda	average
15	15	49	Rebecca	average

Рис. 41: Результат для заполнения таблицы “animal”

- Таблица aviary

```
num_aviaries = 10000
aviary_ids = []
for _ in range(num_aviaries):
    cur.execute(
        "INSERT INTO aviary(id_shelter, type, capacity) VALUES (%s, %s, %s) RETURNING id_aviary",
        (random.choice(shelter_ids), random.choice(["dog", "cat"]), random.randint(1, 4))
    )
    aviary_ids.append(cur.fetchone()[0])
conn.commit()
```

Рис. 42: Код для заполнения таблицы “aviary”

	id_aviary [PK] integer	id_shelter integer	type character varying (10)	capacity integer
1	1	5	dog	2
2	2	3	dog	1
3	3	7	dog	1
4	4	2	dog	2
5	5	1	dog	4
6	6	4	cat	3
7	7	1	dog	4
8	8	6	cat	2
9	9	4	cat	1
10	10	2	cat	1
11	11	6	cat	3
12	12	1	cat	1
13	13	7	dog	3
14	14	6	dog	1
15	15	1	dog	3
16	16	2	dog	1
17	17	7	cat	4

Рис. 43: Результат для заполнения таблицы “aviary”

- Таблица shelter

```
num_shelters = 100
shelter_ids = []
for _ in range(num_shelters):
    cur.execute(
        "INSERT INTO shelter(name, address) VALUES (%s, %s) RETURNING id_shelter",
        (fake.company(), fake.address())
    )
    shelter_ids.append(cur.fetchone()[0])
conn.commit()
```

Рис. 44: Код для заполнения таблицы “shelter”

	id_shelter [PK] integer	name character varying (100)	address text
1	1	Delacruz-Randall	402 Kramer Inlet
2	2	Harris-Austin	680 Thompson Extensions
3	3	Trevino-Maxwell	9578 Michael Vista Suite 052
4	4	Freeman, Velasquez and Bla...	PSC 5018, Box 0571
5	5	Hernandez, Ramos and Moo...	PSC 4922, Box 0309
6	6	Mccoy-Jackson	223 Brittany Street
7	7	Jones-Davis	11924 George Garden Suite 064

Рис. 45: Результат для заполнения таблицы “shelter”

6. Выполнение запросов

В этом разделе приведены различные запросы к реализованной базе данных — их краткие описания, непосредственно запрос на языке SQL и результат выполнения.

1. Вывести животных, у которые болеют

```
select * from animal
where health='sick'
```

Запрос 1. Рис. 46

	id_animal [PK] integer	id_aviary integer	name character varying (100)	health character varying (10)
1	4	4	Thomas	sick
2	8	10	[null]	sick
3	13	14	[null]	sick
4	14	15	Austin	sick

Результат запроса 1. Рис 47

2. Получить список всех работников приюта с заработной платой больше заданной суммы.

```
select full_name from workers
where salary > 10000
```

Запрос 2. Рис. 48

	full_name character varying (100) 🔒
1	Robert Vazquez
2	Donald Hughes
3	Tiffany Cooper
4	Sophia Baker
5	Nicholas Pena

Результат запроса 2. Рис 49

3. Найти всех работников чье имя начинается с буквы А

```
select * from workers
where full_name ~ '^A'
```

Запрос 3. Рис. 50

	id_worker [PK] integer 🔒	id_shelter integer 🔒	full_name character varying (100) 🔒	passport integer 🔒	birthday date 🔒	salary integer 🔒	sex character varying (1) 🔒
1	27	36	Andrew Mercado	399199	1992-08-...	32584	M
2	43	46	Audrey Hart	615364	1978-02-...	62776	F
3	61	39	Albert Hardy	645167	1984-03-...	31669	M
4	66	37	Anthony Mckinney	184507	1996-09-...	42788	M
5	67	48	Amanda Warren	353041	1982-12-...	32334	F

Результат запроса 3. Рис 51

4. Найти минимальную и максимальную вместимость вольеров.

```
select min(capacity) as min_c, max(capacity) as max_c from aviary
```

Запрос 4. Рис. 52

	min_c integer 🔒	max_c integer 🔒
1	1	4

Результат запроса 4. Рис 53

5. Посчитать общее количество пожертвований в базе данных.

```
select count(id_donate) as counts from donates
```

Запрос 5. Рис. 54

	counts bigint
1	10000

Результат запроса 5. Рис 55

6. вывести работников со списком премий

```
select full_name ,
case when salary<50000 then 10000
else 20000 end as award from workers
```

Запрос 6. Рис. 56

	full_name character varying (100)	award integer
1	Robert Vazquez	10000
2	Donald Hughes	20000
3	Tiffany Cooper	20000
4	Sophia Baker	20000
5	Nicholas Pena	10000

Результат запроса 6. Рис 56

7. Получить список животных вместе с названием приюта, где они находятся.

```
select animal.name, shelter.name from animal
join aviary on animal.id_aviary=aviary.id_aviary
join shelter on shelter.id_shelter=aviary.id_shelter
```

Запрос 7. Рис. 57

	name character varying (100)	name character varying (100)
1	Keith	Cook-Wilson
2	Michelle	Smith-Garcia
3	Rebekah	Smith-Garcia
4	Thomas	Smith-Garcia
5	Anne	Smith LLC
6	Michael	Cook-Wilson

Результат запроса 7. Рис 58

8. Найти количество животных в каждом вольере.

```
select id_aviary, count(*) from animal
group by id_aviary
```

Запрос 8. Рис. 59

	id_aviary integer	count bigint
1	1489	3
2	4790	1
3	273	4
4	6977	1
5	3936	2

Результат запроса 8. Рис 60

9. Показать работников с указанием названия приюта, где они работают, и отсортировать по зарплате по убыванию.

```
select workers.*, shelter.name from workers
join shelter on shelter.id_shelter=workers.id_shelter
order by salary desc
```

Запрос 9. Рис. 61

	id_worker integer	id_shelter integer	full_name character varying (100)	passport integer	birthday date	salary integer	sex character varying (1)
1	6069	21	Michelle Turner	760514	1979-11-...	89993	F
2	3546	2	Jane Rios	219940	1999-12-...	89987	F
3	1945	34	Veronica Brewer	209642	1996-07-...	89970	F
4	2153	4	Ryan Jacobs	493915	1989-09-...	89967	M
5	4454	24	David Phillips	578987	1981-08-...	89957	M

Результат запроса 9. Рис 62

10. Найти всех благотворителей, сделавших пожертвование и показать сумму их пожертвований.

```
select donates.id_benefactor, benefactor.full_name, sum(donates.money) from donates
join benefactor on benefactor.id_benefactor=donates.id_benefactor
group by donates.id_benefactor, benefactor.full_name
```

Запрос 10. Рис. 63

	id_benefactor integer	full_name character varying (200)	sum bigint
1	551	David Mccoy	1545
2	1704	Susan Taylor	17734
3	6941	Andrew Alvarado	12660
4	8428	Linda Hayes	1825
5	4284	Patricia Smith	593

Результат запроса 10. Рис 64

11. Найти сбор, который принёс наибольшую общую сумму пожертвований, с использованием оконной функции.

```
select id_fundraiser, total_money, rank
from (
    select id_fundraiser, sum(money) as total_money, dense_rank() over (order by sum(money) desc) as rank
    from donates
    group by id_fundraiser
)
where rank = 1
```

Запрос 11. Рис. 65

	id_fundraiser integer	total_money bigint	rank bigint
1	3869	142471	1

Результат запроса 11. Рис 66

12. Найти животных, которые никогда не проходили лечение.(3 способа)

а. 1 способ

```
select animal.name, animal.id_animal from animal
where animal.id_animal not in (select treatment.id_animal from treatment)
```

Запрос 12. Способ 1. Рис. 67

	name character varying (100)	id_animal [PK] integer
1	Michelle	2
2	Michael	6
3	Denise	7
4	[null]	8
5	Matthew	12

Результат запроса 12. Способ 1. Рис 68

b. 2 способ

```
select animal.name, animal.id_animal from animal
where not exists (
    select * from treatment
    where treatment.id_animal = animal.id_animal
)
```

Запрос 12. Способ 2. Рис. 69

	name character varying (100)	id_animal [PK] integer
1	Anna	2848
2	Dorothy	251
3	Ryan	10295
4	Andre	6158
5	Lori	5411

Результат запроса 12. Способ 2. Рис 70

c. 3 способ

```
select animal.name, animal.id_animal from animal
left join treatment on animal.id_animal = treatment.id_animal
where treatment.id_animal is null;
```

Запрос 12. Способ 3. Рис. 71

	name character varying (100)	id_animal [PK] integer
1	Anna	2848
2	Dorothy	251
3	Ryan	10295
4	Andre	6158
5	Lori	5411

Результат запроса 12. Способ 3. Рис 72

13. Вывести информацию о лечении животных за последний месяц

```
select animal.name, count(treatment.id_treatment), max(treatment.start) from animal
join treatment on treatment.id_animal=animal.id_animal
where start > NOW() - INTERVAL '1 month'
group by animal.id_animal, animal.name
```

Запрос 13. Рис. 73

	name character varying (100)	count bigint	max timestamp without time zone
1	Alexandria	1	2025-11-10 00:00:00
2	Mackenzie	1	2025-12-06 00:00:00
3	Erin	1	2025-11-13 00:00:00
4	Samuel	1	2025-11-25 00:00:00
5	Steven	1	2025-11-17 00:00:00

Результат запроса 13. Рис 74

14. нахождение животных в клинике по дням в течении 2 месяцев

```
with recursive days as (
    select id_animal, start, "end" as end_day from treatment
    union all

    select id_animal, start + INTERVAL '1 day', end_day from days
    where start < end_day
),
daily_counts as (
    select start, count(id_animal) as animals_treated from days
    where start >= CURRENT_DATE - INTERVAL '1 months'
    group by start
)

select * from daily_counts
order by start;
```

Запрос 14. Рис. 75

	start timestamp without time zone	animals_treated bigint
1	2025-11-08 00:00:00	43
2	2025-11-09 00:00:00	41
3	2025-11-10 00:00:00	47
4	2025-11-11 00:00:00	47
5	2025-11-12 00:00:00	48

Результат запроса 14. Рис 76

15. удалить работника с id 1

```
DELETE FROM workers
WHERE id_worker=1
```

Запрос 15. Рис. 77

16. смена статуса вылеченным за месяц животным

```
UPDATE animal
SET health = 'healthy'
WHERE id_animal IN (
    SELECT treatment.id_animal FROM treatment
    WHERE treatment.end >= CURRENT_DATE - INTERVAL '1 month'
)
```

Запрос 16. Рис. 78